

Appendix — Herat Heritage Watch

research/APPENDIX.md · branch research · commit 44dc574 · rendered 2026-09-23
<https://github.com/whamidou006/heritage-watch-release>

Supporting ablations for `REPORT.md`. All experiments use the **merged** representation (DINOv2 + Satlas-SI, 8064-d), spatially-blocked CV and in-fold preprocessing.

Protocol note. §A1–A3 were measured with the original 3-seed protocol before we discovered that reseeding `StratifiedGroupKFold` barely changes the partition (§A5). Their **point estimates are unaffected**, but their $\pm$ values understate uncertainty: read every delta in A1–A3 against the corrected **± 0.016** noise floor. This does not change any conclusion in those sections, because each is a *null* result (the measured effects are smaller than the noise floor) or a *large* effect (well outside it).

A1 • Impact of class balance

The label set is skewed 6.8 : 1 (New Construction 352 → Temporary Structure 52). We measured the effect rather than assuming it.

(i) Loss reweighting is a no-op

Refitting the identical head with `class_weight=None` (`scripts/balance_study.py`):

Class	n	F1 balanced	F1 unweighted	$\Delta F1$	Recall bal.	Recall unw.	ΔR
New Construction	352	0.783	0.790	−0.007	0.824	0.845	−0.021
Solar Panel	308	0.919	0.921	−0.002	0.909	0.911	−0.002
Destruction	126	0.455	0.450	+0.005	0.413	0.397	+0.016
Temporary Structure	52	0.671	0.671	+0.000	0.628	0.590	+0.038
Macro-F1		0.707	0.708	−0.001			

Balancing is worth **−0.001 macro-F1**, well inside the ± 0.004 seed spread. It shifts recall toward the minority classes (+0.038 Temporary, +0.016 Destruction) and away from the majority (−0.021), but this is a recall *transfer*, not a gain.

(ii) What imbalance actually controls

Downsampling New Construction, everything else fixed (`scripts/size_study.py`):

n(New Construction)	F1 New Construction	F1 Destruction	Macro-F1
352 (full)	0.783 $\pm$ 0.005	0.455 $\pm$ 0.015	0.707
250	0.706 $\pm$ 0.016	0.486 $\pm$ 0.036	0.691
180	0.667 $\pm$ 0.020	0.514 $\pm$ 0.018	0.701
126 (matched)	0.597 $\pm$ 0.027	0.620 $\pm$ 0.018	0.710
90	0.528 $\pm$ 0.036	0.670 $\pm$ 0.003	0.711

n(New Construction)	F1 New Construction	F1 Destruction	Macro-F1
52	0.385 ±0.067	0.682 ±0.027	0.691

- **Destruction is not intrinsically hard.** At matched support (126 vs 126) it scores **0.620 vs 0.597** — marginally *better* than New Construction. Its 0.455 in the full setting is a consequence of being outnumbered 2.8 : 1 by the class it is confusable with.
- **Imbalance redistributes rather than destroys.** Macro-F1 stays flat (0.691–0.711) across a 6.8× change in support; the two directional classes trade F1 almost one-for-one.
- **Absolute class size does not predict difficulty.** Temporary Structure (n = 52) outscores Destruction (n = 126) by +0.22 F1 because it is not confusable with a large class. The apparent $\text{corr}(\text{F1}, \text{class size}) = +0.69$ over four points is an artefact; the governing quantity is the **support ratio against confusable classes**.

Implication. Collecting more Destruction labels would raise Destruction F1 largely at New Construction's expense, leaving macro-F1 near 0.71. Breaking that ceiling requires better *directional* features, not a rebalanced dataset.

A2 • Impact of dataset cleaning

Four filters are applied. Two are hard constraints (no pixels exist): *outside footprint* (367) and *endpoint year unimaged* (134). The other two are **choices**, measured in `scripts/cleaning_study.py`:

Condition	n	classes	Macro-F1	vs cleaned
A. Cleaned (reported)	838	4	0.7071 ±0.004	—
B1. Raw, naive 6-class macro	864	6	0.5792 ±0.000	−0.128
B2. Raw, scored on the 4 target classes	864	6	0.6996 ±0.002	−0.008
C. Deduplicated, 1 label per 32 m	418	4	0.6853 ±0.006	−0.022

The B1→B2 gap of 0.128 is a metric artefact, not a model effect. `Reconstruction` has a single sample, scores F1 = 0.000 by construction, and under macro-averaging over six classes removes ~0.13 on its own. Excluding it is a *reporting* decision, not a modelling one.

Keeping the discarded classes costs almost nothing — as distractors they cost −0.008 on the four targets, inside noise:

Class	Cleaned	Raw (6-class)	Δ
Destruction	0.455	0.467	+0.013
New Construction	0.783	0.765	−0.018
Solar Panel	0.919	0.904	−0.015
Temporary Structure	0.671	0.662	−0.009
Other (n = 25)	—	0.677	—
Reconstruction (n = 1)	—	0.000	—

`Other` reaches **F1 0.677** — it is *not* an unlearnable grab-bag and likely contains recoverable structure worth re-annotating.

The headline does not rest on overlapping chips. Chips are 32 m wide, so labels closer than that share pixels: 1,684 such pairs exist, but only **87 (5 %) cross a spatial-block boundary** — the blocking already removes 95 % of chip-overlap leakage. Enforcing one label per 32 m discards **half the data (838 → 418) for only −0.022 macro-F1**, confirming the score is not an artefact of near-duplicate patches.

A3 • Impact of chip size

Chip size was the one free hyperparameter, fixed a priori at 128 px (`scripts/chip_study.py`):

Chip	Ground extent	n	Macro-F1
32 px	8 m	842	0.6789 ±0.004
64 px	16 m	841	0.7207 ±0.000
128 px	32 m	838	0.7071 ±0.004
256 px	64 m	832	0.6175 ±0.017

A clear inverted-U: too small and the object is not fully contained; too large and the labelled change is diluted by unrelated context, costing **−0.09**. This range rivals the gap between backbones.

Under the corrected paired protocol (§A5), **64 px vs 128 px is not resolved**: +0.019 mean, sd 0.022, winning 7 of 8 replicates. The *extremes* (256 px is bad, 32 px is bad) are unambiguous; the choice between 64 and 128 is not.

Caveat. 64 px was also selected using the same CV that scores it, so 0.721 is doubly optimistic. The protocol-clean headline remains **0.711 at 128 px**, and all other ablations are run at 128 px for comparability.

Per-class, however, the difference is concentrated and interpretable — smaller chips help precisely the class that fails:

Class	128 px	64 px	Δ
Destruction	0.455	0.577	+0.122
New Construction	0.783	0.803	+0.020
Solar Panel	0.919	0.911	−0.008
Temporary Structure	0.671	0.592	−0.079

Destruction's confusion into New Construction falls from 46.8 % to 35.7 %. This is consistent with §A1: a tighter crop removes surrounding intact context that makes a demolished footprint resemble a construction site. The cost falls on Temporary Structure, which apparently needs context to be recognised.

A4 • The `Other` class and the date shortcut

What the 25 `Other` labels are

The CSV carries no free-text description, but the provenance columns are revealing: **20 of the 25 `Other` labels come from a single annotation layer** (`changebetween_202310_and_202408`).

Acquisition pair	Other labels
2023/2024	20
2017/2018	2
2024/2025	2
2022/2023	1

This is an **annotation-session artefact**, not a semantic category: `Other` is largely "whatever one annotator flagged in one pass". That it still reaches F1 0.677 (§A2) is therefore suspicious, and motivated the following control.

Class × acquisition pair

Class	09/10	12/13	13/16	16/17	17/18	18/21	21/22	22/23	23/24	24/25	n
Destruction	3	3	45	15	8	16	2	5	15	14	126
New Construction	31	20	93	32	26	64	34	23	15	14	352
Solar Panel	0	0	0	0	0	24	16	136	67	65	308
Temporary Structure	7	4	3	1	1	0	2	33	1	0	52
Other	0	0	0	0	2	0	0	1	20	2	25
Reconstruction	0	0	0	0	1	0	0	0	0	0	1

Solar Panel does not exist before 2018 — a genuine historical fact, not an annotation error, but one that makes the acquisition date partially diagnostic. Temporary Structure is 33/52 in a single pair.

Imagery-free control

Predicting the class from the **one-hot acquisition pair alone**, no pixels (`scripts/perclass_shortcut.py`, 6-class raw set):

Features	Macro-F1
Majority-class floor	0.097
Date one-hot only (no imagery)	0.272
Imagery (merged)	0.579
Imagery + date	0.582

A date shortcut exists but is modest. Date alone reaches 2.8× the floor, so part of the apparent skill on the date-concentrated classes (Solar Panel, Temporary Structure, `Other`) is temporal rather than semantic. Reassuringly, **adding the date to imagery buys only +0.003**, indicating the imagery model has already absorbed whatever the date provides rather than being blocked from it.

The naive rule "pair = 2023/2024 ⇒ `Other`" yields P 0.169 / R 0.800 / F1 0.280 — far below the 0.677 the model achieves, so `Other` is not *purely* a date signature either. It plausibly contains real recurring content worth re-annotating.

Recommendation for the hackathon. Either re-annotate `Other` into the taxonomy, or exclude it and state why. Whichever is chosen, report per-class results so date-driven classes cannot flatter an aggregate score.

A5 • Corrected noise floor (protocol fix)

While reviewing the per-class tables we noticed the 64 px configuration reported ± 0.000 across all classes. It was not stability:

- `StratifiedGroupKFold` assigns *groups* to folds greedily; `shuffle/random_state` only affects tie-breaking.
- On the 64 px set ($n = 841$) it returned the **identical partition for all 3 seeds** — 0 of 841 predictions differed.
- At 128 px only **27 of 838** predictions differed across seeds.

So the "3 seeds" were close to a single measurement, and the resulting ± 0.002 – 0.006 was not a noise estimate. **Fix:** jitter the grid origin by a random fraction of a block per replicate (`scripts/noise_floor.py`), which moves every block boundary and produces genuinely different, still spatially blocked, partitions.

Config	n	mean	sd	min	max
64 px	841	0.7302	0.0123	0.7092	0.7456
128 px	838	0.7112	0.0162	0.6897	0.7446

The true single-configuration noise floor is ± 0.016 — roughly 4× the originally reported value.

Re-running the full comparison over 8 **paired** jittered grids (`scripts/compare_jitter.py`):

Representation	dim	mean	sd	Paired Δ vs best	wins	verdict
DINOv2, post-event	768	0.5818	0.016	−0.1294	8/8	resolved
DINOv2, both + diff	2304	0.6166	0.014	−0.0946	8/8	resolved
Satlas-SI, post-event	1920	0.6355	0.014	−0.0757	8/8	resolved
Satlas-SI, both + diff	5760	0.6813	0.010	−0.0299	8/8	resolved
Satlas-MI fused	1920	0.6479	0.014	−0.0633	8/8	resolved
Satlas-MI + SI diff	3840	0.6902	0.016	−0.0210	8/8	resolved
Merged (selected)	8064	0.7112	0.016	—	—	—

Every ranking in the report survives, and the ordering is unchanged. But it survives *because of pairing*: the two closest gaps (0.021 and 0.030) are smaller than the ± 0.016 single-run noise and could not have been separated by comparing independent means. Any future claim below ~ 0.03 on this dataset must be tested paired.

A6 • Hackathon pack

Data to make available

Artefact	Contents	Size
herat_site_TM_z19/	21 co-gridded GeoTIFF scenes, EPSG:4326, ≈ 0.25 m/px, 2009–2025	~ 3 GB
Herat_all_changes.csv	1,370 annotated points: fid, category, X, Y, year	small
shortlist_sites.geojson	8 World Heritage site polygons (context / reporting)	small
cache/manifest.json	838 pre-filtered labels with resolved T1/T2 scene paths	small

Artefact	Contents	Size
cache/features*.npz	Precomputed DINOv2 + Satlas embeddings (838 × 768 and × 1920)	~30 MB
out/*.json	Baseline numbers for every ablation above	small

Ship the precomputed embeddings. They reproduce the 0.711 baseline in under a minute on CPU, so no team loses hackathon hours to backbone downloads or GPU queues. Shipping `manifest.json` also guarantees every team evaluates on an identical label set.

Two gotchas in the raw data

1. The `x` column header has a **trailing space** (`"x "`) — a silent `KeyError` source.
2. Coordinates and `.tfw` sidecars use **comma decimal separators** (French locale); parse with `float(s.replace(",", "."))`. The GeoTIFFs carry correct internal CRS, so only sidecar parsing is affected.

Rules we recommend enforcing

1. **Spatially-blocked CV is mandatory; random CV is disqualifying.** It inflates scores by 2.5–6.3 pp, so mixed protocols make a leaderboard meaningless. Publish `spatial_blocks()` and fixed fold seeds as starter code.
2. **Report macro-F1 over the 4 classes, plus the per-class table.** Accuracy is unusable at 42 % / 6 % imbalance, and a single aggregate hides the Destruction failure.
3. **Fit every transform inside the fold** (scaler, PCA, class weights).
4. **Report 3 seeds with a spread.** Several effects here are smaller than the seed spread; without $\pm$, teams will report noise as improvement.

Suggested baselines and difficulty

Tier	Task	Expected macro-F1
Starter	Majority-class predictor	0.148
Easy	DINOv2 post-event image + logistic head	~0.58
Baseline	Merged DINOv2 + Satlas-SI (provided)	0.711
Strong	Tune chip size / fine-tune an encoder	~0.72
Winning	Beat 0.55 F1 on Destruction without losing New Construction	open

Where the real problem is

Point teams at **Destruction (F1 0.476)** and give them §A1 so they do not waste time on dead ends: **rebalancing will not help** (−0.001), and **more labels mainly shift F1 between Destruction and New Construction** rather than raising the total. The open problem is representing the *direction* of change — an encoder that is not temporally order-invariant, an explicit ordered-pair model, or synthetic ordered pairs (Changen2 / ChangeStar2).

Starter commands

```
python scripts/build_manifest.py          # 1,370 -> 838 labels
python scripts/embed_chips.py             # DINOv2 (skip if embeddings shipped)
python scripts/embed_satlas.py            # Satlas (skip if embeddings shipped)
python scripts/compare_gpu.py --pca 0     # reproduces the REPORT.md table

# ablations
python scripts/balance_study.py           # A1(i)
python scripts/size_study.py              # A1(ii)
python scripts/cleaning_study.py          # A2 (needs --keep-all manifest, see
below)
python scripts/chip_study.py               # A3 (needs --chip {32,64,256}
embeddings)
python scripts/perclass_shortcut.py        # A4 per-class + date-shortcut control
python scripts/noise_floor.py             # A5 corrected noise floor
python scripts/compare_jitter.py          # A5 paired re-run of the main table
```

Building the uncleaned variant used in A2:

```
python scripts/build_manifest.py --keep-all --out manifest_raw.json
python scripts/embed_chips.py --manifest manifest_raw.json --out features_raw.npz
python scripts/embed_satlas.py --manifest manifest_raw.json --out features_satlas_raw.npz
```

A7 · HASTE control: is a single-date model enough for the damage class?

`microsoft/haste` (MIT, Microsoft AI for Good) is the platform this work would be delivered into, so its modelling assumptions matter. Reading the source rather than the docs:

Component	What HASTE does	Source
Labeling features	Frozen MOSAICS (torchgeo RCF) or DINOv2 ViT-S/B/L patch tokens on per-building chips	<code>hastelib/.../processors/embedding.py</code>
Assessment model	U-Net-style semantic segmentation with <code>constraint_segmentation_loss</code> , trained on Azure Batch	<code>docker/training/code/fine_tune.py</code> , <code>bda/trainers.py</code>
Temporal input	Single date. <code>num_channels: 3 # (3 for RGB)</code> ; the datamodule appends a one-element list per sample	<code>docker/training/code/configs/config.yml</code> , <code>bda/datamodules.py:60</code>
Output	Per-pixel damage aggregated onto OSM/Overture footprints → GeoPackage	<code>merge_with_building_footprints.py</code>

HASTE ingests pre- *and* post-event imagery, but only the **post-event** date reaches the network; the pre-event scene feeds the side-by-side visualiser. `TRANSPARENCY.md` agrees.

The first two rows independently validate our architecture: HASTE's labeling path is frozen-DINOv2-plus-light-head on object chips, which is what we built. The third row predicts a failure, and we measured it rather than asserting it.

Setup

Same frozen embeddings, same paired jittered spatial protocol (8 replicates, §A5). Only the **temporal input** varies, so nothing else can explain a difference:

Representation	Input	Meaning
pre_only	f1	before-image alone
haste_post	f2	after-image alone — the faithful HASTE analogue
diff	f2 - f1	direction without absolute appearance
bitemporal	[f1, f2, f2 - f1]	our system

Task A — Destruction vs New Construction (n = 478, 126 positive)

These two classes are the *same two images in the opposite order*, so this is the sharpest possible test. Prevalence baseline AP = 0.264.

Representation	DINOv2 F1	DINOv2 AP	Satlas F1	Satlas AP
pre_only	0.317 ± 0.019	0.308 ± 0.017	0.315 ± 0.028	0.316 ± 0.018
diff	0.405 ± 0.027	0.428 ± 0.025	0.466 ± 0.032	0.499 ± 0.027
haste_post	0.433 ± 0.015	0.485 ± 0.015	0.518 ± 0.019	0.528 ± 0.016
bitemporal	0.416 ± 0.021	0.451 ± 0.030	0.469 ± 0.028	0.478 ± 0.018

Paired bitemporal - haste_post: DINOv2 **-0.017 F1 (0/8)**, -0.033 AP (0/8); Satlas **-0.048 F1 (0/8)**, -0.050 AP (0/8). The single-date representation wins **8/8 on both encoders and both metrics**. Under the three-bar rule of §4, **three of the four contrasts are resolved**; the DINOv2 F1 arm (0.0167) is unanimous and beats 2-SE but falls below the 0.02 minimum effect size, so it counts as consistent-but-small rather than resolved.

Task B — full 4-class

Representation	DINOv2 macro	Destruction	Satlas macro	Destruction
pre_only	0.579	0.261	0.570	0.235
diff	0.450	0.337	0.568	0.429
haste_post	0.581	0.356	0.632	0.442
bitemporal	0.613	0.360	0.673	0.418

HASTE control

What this changes

- A single-date model is not handicapped on damage — it is the best option we tested.** Rubble is directly visible in the after-image; the classifier does not need the date order to recognise it. Adding the before-image and the difference triples the dimensionality and costs 0.017–0.048 F1 on this pair.
- This does not contradict the Satlas-MI result (finding 3).** Satlas-MI max-pools over both dates, producing a function that is *symmetric* in the pair and provably cannot encode direction (swapping inputs changes the output by exactly 0.0). Using the post-event image *specifically* is not symmetric — selecting the later date is itself directional information. Order-invariance is harmful; single-date is not.
- Bi-temporal input still earns its place overall** (+0.032 DINOv2, +0.041 Satlas macro-F1), but the gain comes from Solar Panel and Temporary Structure, **not** from the damage class.

4. `pre_only` **is near the prevalence floor** (AP 0.308 / 0.316 vs 0.264), which rules out the alternative explanation that `haste_post` merely benefits from better image quality or a date artefact in the later scenes.

Consequence for delivery. HASTE can be adopted as the labeling front-end, visualiser and GeoPackage export path without its single-date design compromising the damage class. The contribution is extending it from binary single-date damage to a **four-class taxonomy** — for which bi-temporal input does help — rather than "fixing" a temporal deficiency it does not have for this class.

Reproduce: `scripts/haste_control.py` → `out/haste_control.json`; figure via `scripts/plot_haste_control.py`.